

root_cause_20260826

BOB-196 — check_cm_plugin_count.sh runtime root cause (investigation only)

Revision: 1 **Last modified:** 2026-08-26T20:34:50Z **Description:** Measured root cause of the CM-PLUGIN-COUNT pre-build gate's runtime, with the prior item text corrected on three points. Investigation only — no source was modified. **Authority:** §11.4.6 (no-guessing), §11.4.50 (repeat measurement), §11.4.102 (systematic debugging), §11.4.201(6)(7)(12) (instrument honesty, control needle, xtrace footgun) **Scope:** scripts/pre_build/check_cm_plugin_count.sh runtime ONLY. Nothing here questions the gate's verdicts — it PASSED (8 documented count(s) match their derivation) on all 12 runs recorded below.

1. Measured wall-clock, with spread and load context

The host was running five other agents throughout; load is recorded per run because the runtime turned out to be load-dependent (§4).

Run	nice	wall (s)	1-min load
1	-n 19	29.394	4.60
2	-n 19	21.734	5.26
3	-n 19	26.667	5.60
4	-n 19	28.895	6.73
5	none	21.729	5.89
6	none	24.191	6.21

Run	nice	wall (s)	1-min load
7	none	41.048	9.63

Spread 21.73 s – 41.05 s, median $\approx$ 26.7 s. nice -n 19 and un-niced runs interleave (21.73/24.19 un-niced vs 21.73/26.67 niced) — under this contention nice produced **no separable effect**; host load dominates. Both were measured, and both are reported, because a timing claim from only the niced set would not be transferable to the un-niced gate run inside pre_build_verification.sh.

Correction to the item’s headline. The item states “189 SECONDS”. That figure is not reproducible as a constant — but it is not wrong either, and it is not stale. The gate is fork-bound, so its runtime is a function of host load (§4):

load	5.26	6.73	9.63	17.06	29.77
wall (s)	21.7	28.9	41.0	47.3	77.4

189 s is the same gate at a heavier load point. The honest statement is a **range governed by contention**, not a single number: ~22 s on a quiet host, ~77 s measured at load 29.8.

2. Root cause — proven, not reasoned

A copy of the gate (probe.sh, differing from the tracked file by exactly one injected line) was traced with PS4='+T:\$ {EPOCHREALTIME}:\${LINENO}:' written to **fd 9 via BASH_XTRACEFD**, not stderr — the §11.4.201(12) countermeasure, since the gate redirects its own streams. Total attributed 90.508 s against a 90.490 s wall (xtrace-inflated run): full accounting.

Instrument offset, disclosed. Trace line numbers are **+1** relative to the tracked file, because the injected instrumentation line sits at 109. A separate control experiment (lineno_control.sh) proved bash itself introduces **no** offset — pipeline elements inside a command substitution trace at their own assignment line. The table below is already corrected to original line numbers. Without that control this investigation would have cited line 285 (if [["\$nmark" -gt 1]], a pure builtin) as a 34-second cost centre — a fabricated finding.

seconds	trace events	original line	source
34.280	11110	284	nmark="\$ (printf '%s' "\$line" \ grep -oE '<!--...CM- PLUGIN-

seconds	trace events	original line	source
20.227	5559	289	COUNT:' \\ wc -l \\ -d ' ' \\\\ true)" metric="\$ (printf '%s' "\$line" \\ sed -n '...CM- PLUGIN- COUNT...')" stated="\$ (printf '%s' "\$line" \\ sed -n '... managed plugins...')" while IFS= read -r line; do (marker loop) while IFS= read -r line; do (legacy loop) if [["\$nmark" -gt 1]]; then
19.341	5559	311	
5.705	3712	278	
5.228	3712	310	
4.642	1853	285	

Six lines = 89.42 s of 90.51 s (**98.8 %**). Phase split:

- lines < 270 — **needle + array counts + find + engines**
derivation: 0.702 s
- lines ≥ 270 — **document parsing loops: 89.806 s (99.2 %)**

The gate spends 0.7 s doing the job it exists for and ~90 s parsing prose.

Process spawns — measured, and the item's arithmetic corrected

16,808 depth-2+ (forked) commands were traced. That decomposes exactly:

- line 284: printf, grep, wc, tr, true = **5** forks/line
- line 289: printf, sed = **2** forks/line

- line 311: printf, sed = **2** forks/line

= **9 forks per governed line × 1853 lines = 16,677**, plus 131 in the derivation phase = 16,808. Exact match.

The item states “~6 [per iteration] ... so ~11,118 process spawns”. That undercounts: it omits the `|| true` fork on line 284 and omits the **entire legacy loop** (line 310–311), which re-reads all three documents a second time. Corrected figure: **~16,700 forks driven by the document loops**.

3. What the gate actually walks vs. what the title says

The item’s title — “runs for over a minute on ~69 files” — attributes the cost to the wrong file set. 69 is the recursive metric (`plugins/**/*.py`), and deriving it is one find inside the 0.702 s derivation phase. **The 69 files are not the cost.**

What the expensive loops actually walk is the three governed documents, every line:

document	lines	CM-PLUGIN-COUNT markers
CLAUDE.md	494	5
AGENTS.md	689	0
docs/features/ Status.md	670	3
total	1853	8

1853 lines are read twice (once per loop) to locate 8 marker lines — a 232:1 ratio. **AGENTS.md contributes 37 % of the scanned volume and zero checkable claims.**

Is that a correctness problem? Checked and **no**: AGENTS.md carries no unmarked plugin-count claims either, so nothing there is silently unguarded. That absence was control-needed — the same pattern on the same path against CLAUDE.md returns its three known claims (`**43 search-plugin engines**`, `**43 distinct engine modules**`, `**12 bootstrap plugins**`), so the zero is real absence, not a blind instrument. A negative control returned 0 as expected. AGENTS.md’s presence in GOVERNED_DOCS is therefore **pure cost, not a latent gap** — it is correct to keep it governed (a future claim added there must be caught), just expensive to scan this way.

Worktree hypothesis re-confirmed dead (independently).
find is at line 224 and is scoped to `PLUGINS_DIR="$ROOT/plugins"`
(line 187). Agent worktrees under `.claude/` are outside that walk
entirely. The BOB-194 refutation holds; nobody should re-walk it.

4. Why it is load-sensitive

The cost is ~16,700 fork+exec pairs, not computation. Process creation contends for the same CPUs as the other agents, so wall-clock scales with load (table in §1) while the derivation phase — which does real work on 69 files — stays flat. This is the signature that distinguishes a fork-bound loop from an algorithmic problem, and it is why a single sample would have been meaningless here.

5. Controlled test of the remediation direction

Identical derivation inputs (plugins/ copied, install-plugin.sh, setup.sh, README.md verbatim), governed documents reduced to **only their marker-bearing lines** (1853 → 8). This is the pre-filter the item proposes, measured without modifying any source file.

	full root	marker-only root	ratio
paired run 1 (load 29.77)	77.355 s	0.749 s	103.3×
paired run 2 (load 17.06)	47.323 s	0.711 s	66.5×
standalone (load 36-38)	—	1.354-2.154 s	—

Verdict **identical in every case**: PASS: CM-PLUGIN-COUNT – 8 documented count(s) match their derivation. The marker-only runtime is also essentially load-independent (0.71 s at load 29.8), confirming §4.

6. Remediation direction (NOT implemented — implementation not authorised)

The item’s candidate fix is **confirmed sound by the measurement above**: pre-filter each governed document with one grep for CM-PLUGIN-COUNT: before entering the marker loop, and

apply the same treatment to the legacy loop's managed plugins shape, leaving the per-line parsing body — including the deliberately-chosen `-oE | wc -l` form — byte-for-byte unchanged.

Three constraints any implementation must respect, from this investigation:

1. **Keep the `-oE | wc -l` form.** The in-code comment at 279-283 records that `grep -coE` returns 3 at top level but 1 inside a set `-euo pipefail` subshell on this host (ugrep 7.8.4) — the §11.4.201(12) footgun. That correctness fix is right; only its *per-line application* is the cost. A pre-filter does not touch it.
2. **The legacy loop must be pre-filtered too**, or roughly a quarter of the cost survives — it is 19.34 s of the 90 s and the item's analysis omits it entirely.
3. **The multi-marker-per-line FAIL path (284-288) must keep biting.** A pre-filter that selects marker lines preserves it (a two-marker line still matches). A pre-filter that instead *extracted* the marker would destroy it, re-opening the §11.4.201(6) false-null the comment at 50-54 was written to close. The existing paired mutation in `tests/pre_build/test_check_cm_plugin_count.sh` is the check that this survived.

Expected result on a quiet host: ~22 s → well under 1 s, verdict unchanged.

7. Wiring — the gate is on the critical path

- `scripts/commit-push-all.sh:229` — stage 3/6 runs `pre_build_verification.sh` (skippable only via the recorded `BOBA_SYNC_SKIP_CI=1` deferral, §11.4.234)
- `scripts/pre_build_verification.sh:1741` — invariant **46**, labelled [46/52], **BLOCKING**
- `tests/pre_build/test_check_cm_plugin_count.sh` — §1.1 paired-mutation meta-test

So with no skip flag it runs on every `commit-push-all.sh`. Its cost is paid in full at T042.

8. Honest boundaries (§11.4.6)

- Timings are from a contended host (5 concurrent agents, load 4.6-38). Absolute numbers do not transfer to a quiet host; the *ratios* in §5 were measured paired at identical load and do.
- The xtrace run is inflated (90.5 s vs ~26 s un-traced). It is used for **relative attribution and fork counting**, both of which the un-traced paired A/B in §5 independently corroborates.

- UNKNOWN: the exact load at which the item's 189 s was originally measured — that run's load was not recorded, so 189 s is explained as a load point but not reproduced.
- Side note, out of scope and **not** verified:
pre_build_verification.sh labels this [46/52], and a comment above it claims "46 labels ... contiguous 1..46" while warning that a second label syntax exists. Counting only the echo "[N/M]" form yields 43 distinct labels against a denominator of 52. The true invariant count requires counting both forms, which was not done here. It is neither 50 nor confirmed to be 52.